Deep Learning

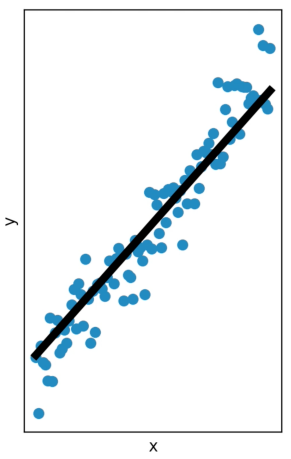
Forest Agostinelli
University of South Carolina

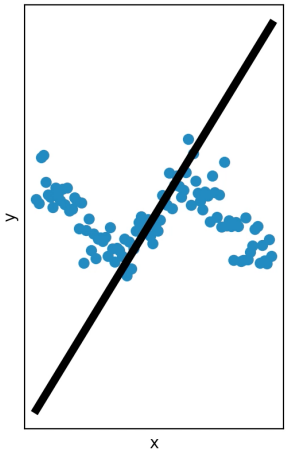
Outline

- Linear models
  - Linear regression
  - Logistic regression
- Neural networks
  - Architecture and backpropagation
  - Deep neural networks
  - Overfitting
  - Optimization

Hypothesis Space

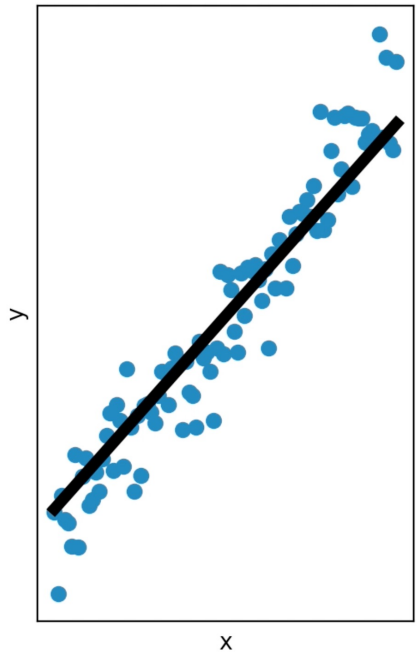
- It is important that the hypothesis space be appropriate for the task at hand
- For example, if the observations have a linear input/output relationship, it is best to use a linear model
- However, if the observations have a non-linear input/output relationship, then a linear model will provide a poor explanation of the data
- On the other hand, if your hypothesis space is too large, then you may learn unnecessarily complicated hypotheses





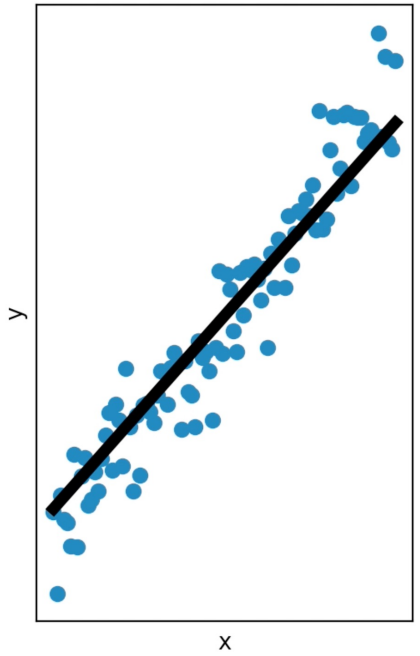
Regression and Classification

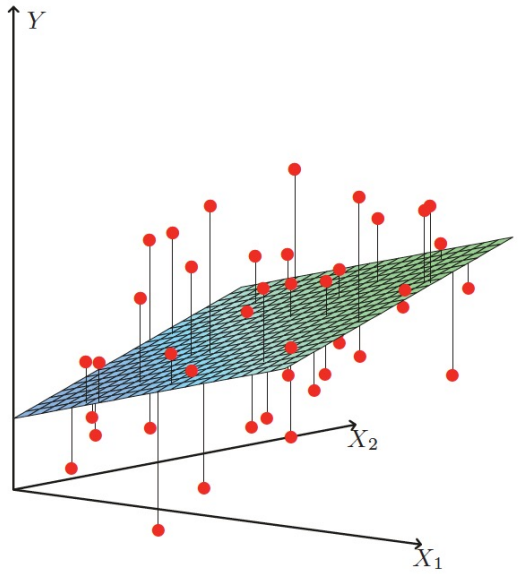
- Learn the relationship between the input $\boldsymbol{x} \in \mathbb{R}^p$ and output $\boldsymbol{y} \in \mathbb{R}^q$
  - $\boldsymbol{y} = f(\boldsymbol{x})$
- The input $\boldsymbol{x}$ is also known as the features or predictors
- **Regression**: $\boldsymbol{y}$ is a continuous variable
- **Classification**: $\boldsymbol{y}$ is a categorical variable



Linear Regression

- Limits model of input/output relationship to a line
- Learning a function $f(\boldsymbol{x}, \boldsymbol{\theta})$ with parameters $\boldsymbol{\theta}$
  - Linear model $\boldsymbol{\theta} = [\boldsymbol{w}, b]$
- $f(\boldsymbol{x}, \boldsymbol{w}, b) = \boldsymbol{w}^T \boldsymbol{x} + b = \sum_i w_i x_i + b$
- Examples (**may not truly be linear!**)
  - Yield of tomatoes as a function of health
  - Expression of a gene as a function of drug concentration





Linear Regression

- Assume 1 dimensional output
- Data
  - Inputs: $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N$ where $\boldsymbol{x}_i \in \mathbb{R}^{p \times 1}$
  - Outputs: $y_1, \ldots, y_N$ where $y_i \in \mathbb{R}$
- Data matrix
  - $\boldsymbol{X} \in \mathbb{R}^{N \times p}$
  - The i^{th} row contains example $\boldsymbol{x}_i$
- Vector of outputs
  - $\boldsymbol{y} \in \mathbb{R}^{N \times 1}$
- Parameters
  - $\boldsymbol{w} \in \mathbb{R}^{p \times 1}$ (weights)
  - $b \in \mathbb{R}$ (biases)
- Loss
  - $\mathcal{L}(\boldsymbol{\theta}) = \sum_n \big(y_n - f(\boldsymbol{x}, \boldsymbol{\theta})\big)^2$

Linear Regression: Analytical Solution

- $\mathcal{L}(\boldsymbol{\theta}) = \sum_n \left(y_n - f(\boldsymbol{x}, \boldsymbol{\theta})\right)^2 = ||\boldsymbol{X}\boldsymbol{w} - \boldsymbol{y}||_2^2$
- $\nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}) = 2\boldsymbol{X}^T(\boldsymbol{X}\boldsymbol{w} - \boldsymbol{y}) = 0$
- $\boldsymbol{w}^* = (\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{X}^T\boldsymbol{y}$

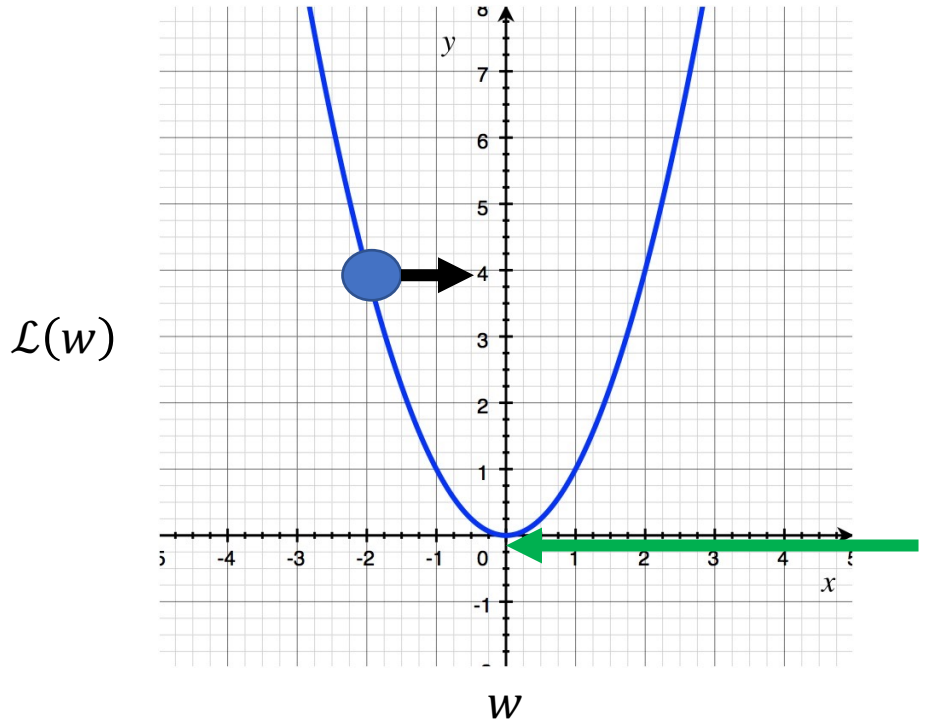
- Say there is no analytical solution, what kind of problem can this be posed as?
  - Optimization problem
  - We can do something similar to hill-climbing search where we want to minimize the loss

Linear Regression: Gradient Descent

- $\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{2n}\sum_n \left(y_n - f(\boldsymbol{x}, \boldsymbol{\theta})\right)^2$
- Gradient – A vector of partial derivatives
  - $\nabla_{\boldsymbol{\theta}}\mathcal{L}(\boldsymbol{\theta}) = [\frac{\partial \mathcal{L}(\theta)}{\partial \theta_0}, \dots, \frac{\partial \mathcal{L}(\theta)}{\partial \theta_{p+1}}]$
  - $\boldsymbol{w} = \boldsymbol{w} - \alpha \nabla_{\boldsymbol{\theta}}\mathcal{L}(\boldsymbol{\theta})$
- Where α is the learning rate
  - This determines how big of a step we take in that direction

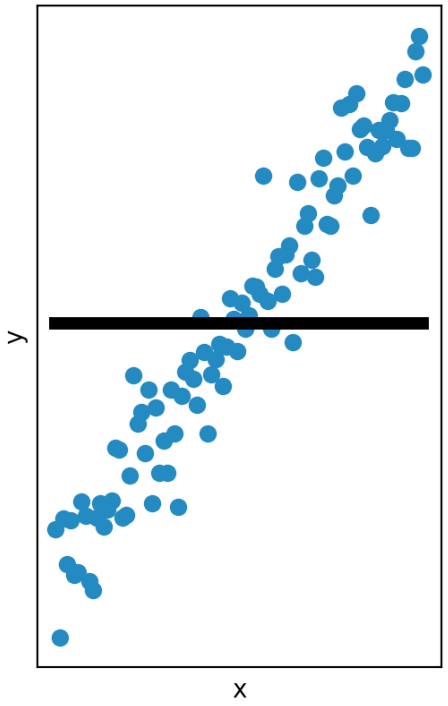
Gradient Descent: 1D Example

- One dimensional example
- $\mathcal{L}(w) = w^2$
- $\frac{\partial \mathcal{L}(w)}{\partial w} = 2w$



Linear Regression: 1D with No Bias

- $y_n = 3x + \epsilon_n$
- $\epsilon_n \sim \mathcal{N}(0,0.5)$
- $f(x_n, w) = wx_n$

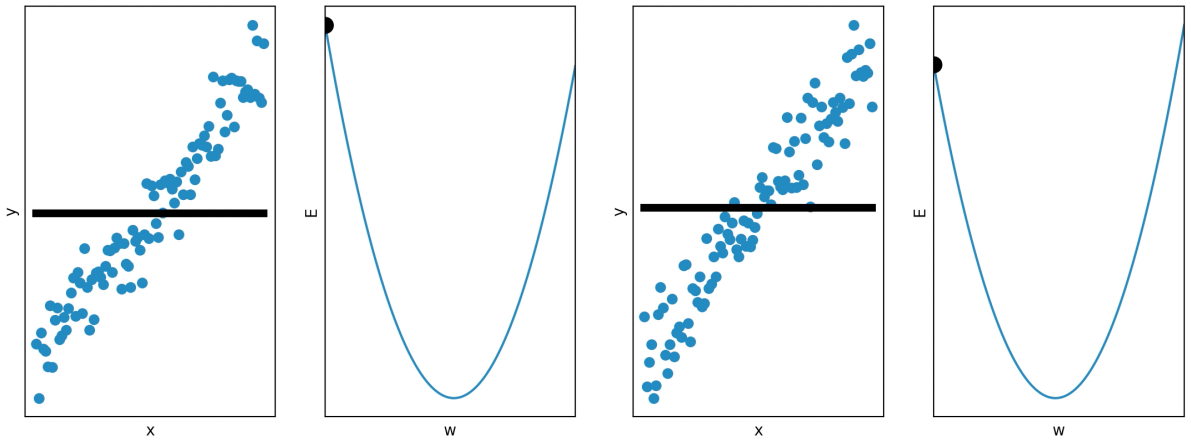


- $\mathcal{L}(w) = \frac{1}{2n}\sum_n (y_n - wx_n)^2$
- What is $\frac{\partial \mathcal{L}(w)}{\partial w}$?
- $\frac{\partial \mathcal{L}(w)}{\partial w} = -\frac{1}{n}\sum_n (y_n - wx_n)\, x_n$

Linear Regression: 1D with No Bias

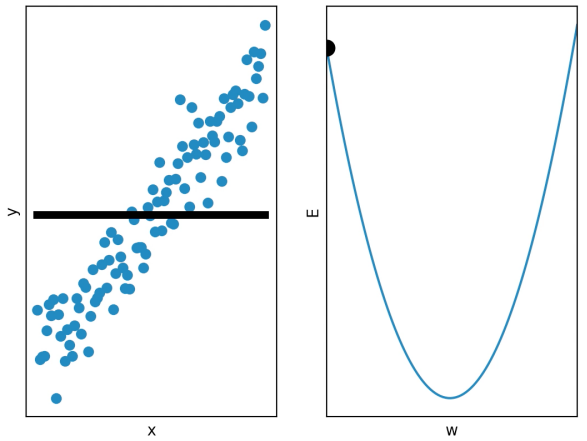
- $y_n = 3x + \epsilon_n$
- $\epsilon_n \sim \mathcal{N}(0,0.5)$
- $f(x_n, w) = wx_n$

- $\mathcal{L}(w) = \frac{1}{2n}\sum_n (y_n - wx_n)^2$
- $\frac{\partial \mathcal{L}(w)}{\partial w} = -\frac{1}{n}\sum_n (y_n - wx_n)\, x_n$
- $w = w - \alpha \frac{\partial \mathcal{L}(w)}{\partial w}$

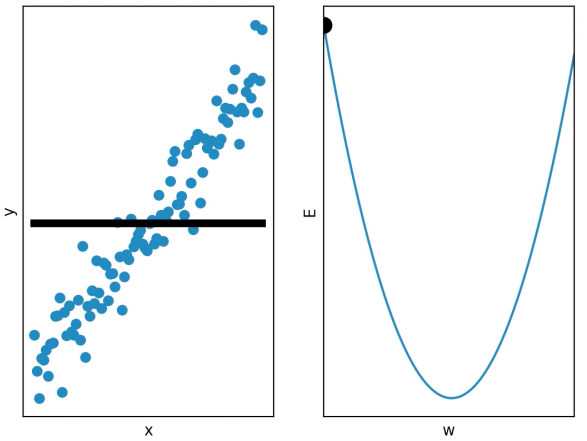


$\alpha = 1.0$

$\alpha = 0.1$



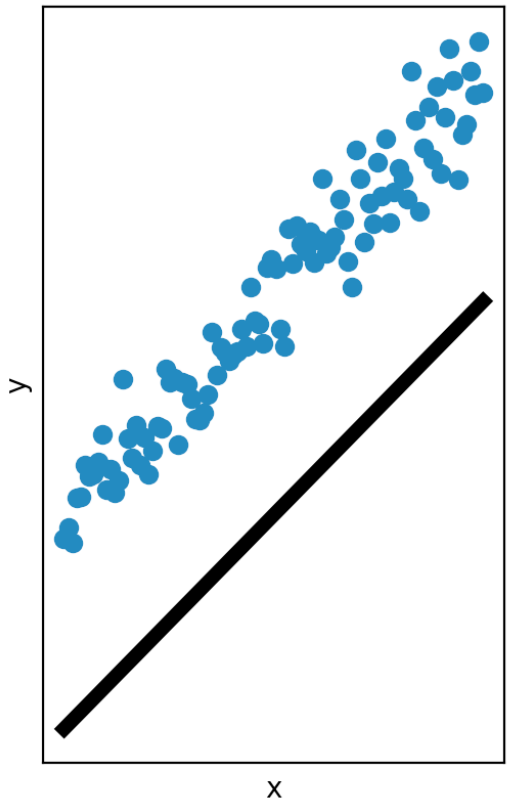
$\alpha = 5.0$



$\alpha = 10.0$

Linear Regression: 1D with Bias

- $y_n = 3x + 3 + \epsilon_n$
- $\epsilon_n \sim \mathcal{N}(0,0.5)$
- $f(x_n, w, b) = wx_n + b$

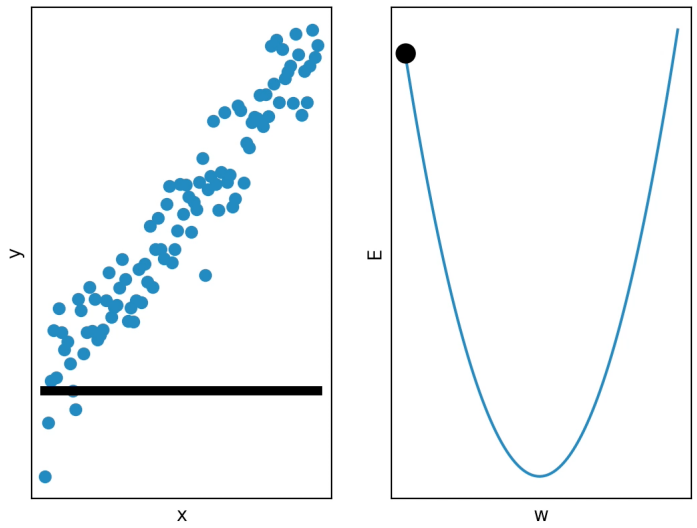


- $\mathcal{L}(w,b) = \frac{1}{2n}\sum_n (y_n - (wx_n + b))^2$
- What is $\frac{\partial \mathcal{L}(w,b)}{\partial w}$ and $\frac{\partial \mathcal{L}(w,b)}{\partial b}$?
- $\frac{\partial \mathcal{L}(w,b)}{\partial w} = -\frac{1}{n}\sum_n (y_n - (wx_n + b))\, x_n$
- $\frac{\partial \mathcal{L}(w,b)}{\partial b} = -\frac{1}{n}\sum_n (y_n - (wx_n + b))$

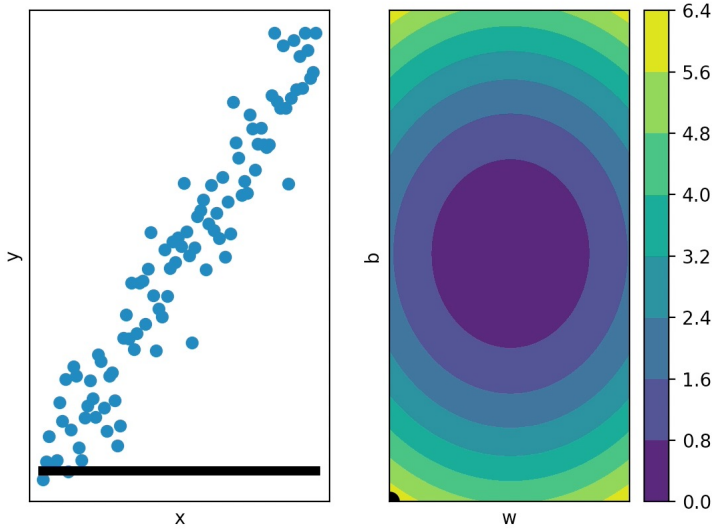
Linear Regression: 1D with Bias

- $y_n = 3x + 3 + \epsilon_n$
- $\epsilon_n \sim \mathcal{N}(0,0.5)$
- $f(x_n, w, b) = wx_n + b$

- $\mathcal{L}(w,b) = \frac{1}{2n}\sum_n (y_n - (wx_n + b))^2$
- $\frac{\partial \mathcal{L}(w,b)}{\partial w} = -\frac{1}{n}\sum_n (y_n - (wx_n + b))\, x_n$
- $\frac{\partial \mathcal{L}(w,b)}{\partial b} = -\frac{1}{n}\sum_n (y_n - (wx_n + b))$
- $w = w - \alpha \frac{\partial \mathcal{L}(w,b)}{\partial w}$
- $b = b - \alpha \frac{\partial \mathcal{L}(w,b)}{\partial b}$



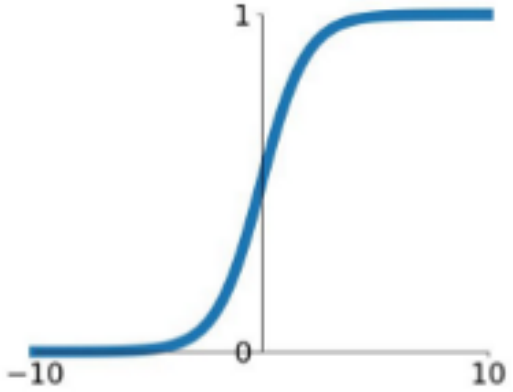
No bias $\alpha = 0.5$



Bias $\alpha = 0.5$

Binary Classification

- We would like to differentiate between 2 classes
  - $P(y = 1|\boldsymbol{x})$
  - $P(y = 0|\boldsymbol{x}) = 1 - P(y = 1|\boldsymbol{x})$
- If values are guaranteed to be positive and have a sum greater than zero, then we can obtain a probability by dividing each value by their sum
  - Ensures normalized values are positive and sum to 1 (obeys the laws of probability)
- We can do this by exponentiating the values $\boldsymbol{w}^T\boldsymbol{x}$
  - $P(y = 1|\boldsymbol{x}) = \frac{e^{w_1^T x}}{e^{w_1^T x} + e^{w_0^T x}} = \frac{1}{1+e^{(w_0 - w_1)^T x}} = \frac{1}{1+e^{-w^T x}}$
- This gives us the logistic function
  - $\sigma(a) = \frac{1}{1+e^{-a}}$
  - $\frac{\partial}{\partial x}\sigma(x) = \frac{\partial}{\partial x}\frac{1}{1+e^{-x}} = \sigma(x)\big(1 - \sigma(x)\big) = \sigma(x)\sigma(-x)$



Likelihood

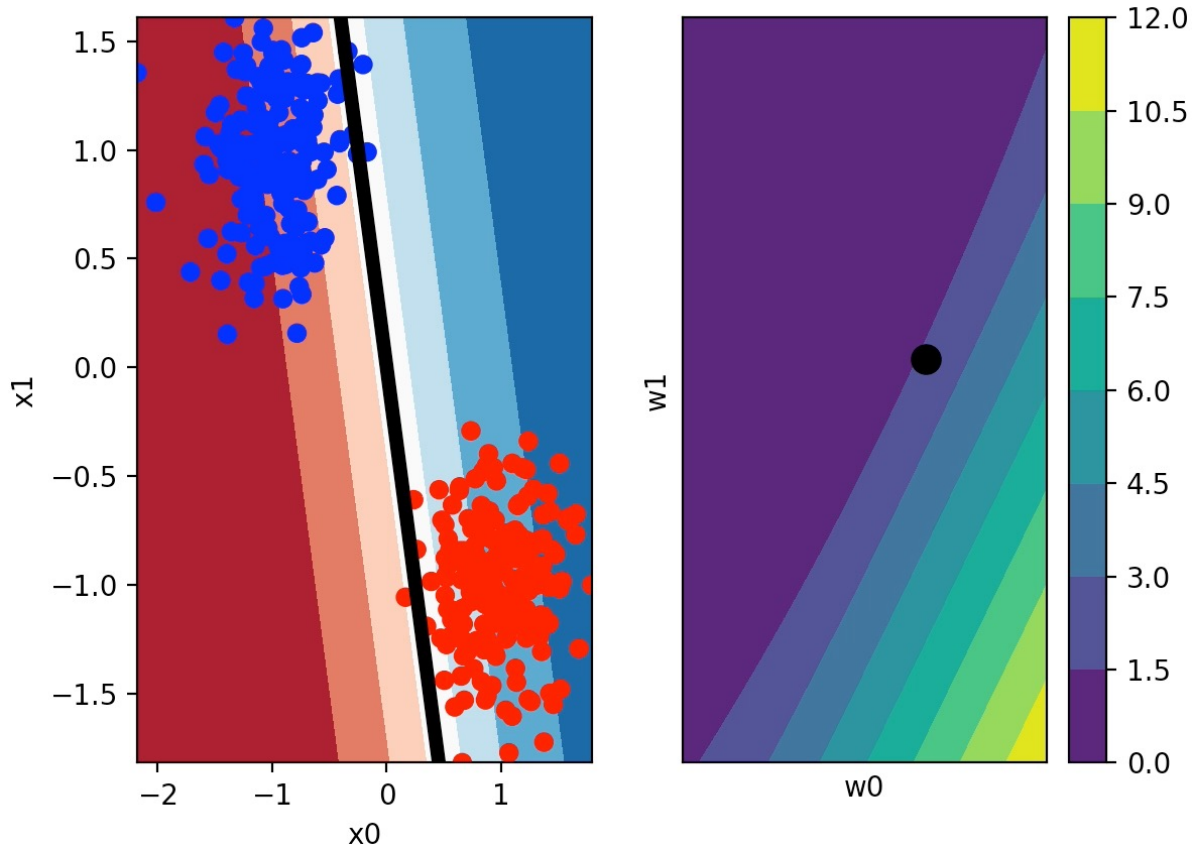
- Likelihood: the joint probability of the observed data given as a function of the parameters of a statistical model
  - Observed data: $((\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \ldots (\boldsymbol{x}_N, y_N))$
  - Parameters: $\boldsymbol{w}$
- $l = \prod_{i=1}^{N} P(y_i | \boldsymbol{x}_i; \boldsymbol{w})$
- $P(y_i | \boldsymbol{x}_i; \boldsymbol{w})$ if $y_i = 1$
  - $\frac{1}{1+e^{-w^T x}}$
- $P(y_i | \boldsymbol{x}_i; \boldsymbol{w})$ if $y_i = 0$
  - $1 - \frac{1}{1+e^{-w^T x}}$

Maximum Likelihood

- We would like to find $\mathbf{w}$ that maximizes the likelihood
- $l = \prod_{i=1}^{N} P(y_i|\boldsymbol{x}_i; \boldsymbol{w})$
- For numerical stability, we take the log of the likelihood
- $ll = \sum_{i=1}^{N} \log P(y_i|\boldsymbol{x}_i; \boldsymbol{w})$
- $= \sum_{i=1}^{N} \mathrm{y_i} \log P(y_i = 1|\boldsymbol{x}_i; \boldsymbol{w}) + (1 - \mathrm{y_i}) \log(1 - P(y_i = 1|\boldsymbol{x}_i; \boldsymbol{w}))$
- $= \sum_{i=1}^{N} \mathrm{y_i} \log(\frac{1}{1+e^{-\boldsymbol{w}^T \boldsymbol{x}}}) + (1 - \mathrm{y_i}) \log(1 - \frac{1}{1+e^{-\boldsymbol{w}^T \boldsymbol{x}}})$
- $\frac{\partial L(\boldsymbol{w})}{\partial w_i} = -\sum_{i=1}^{N} \big(y - \sigma(\boldsymbol{w}^T \boldsymbol{x})\big) x_i = \sum_{i=1}^{N} (\sigma(\boldsymbol{w}^T \boldsymbol{x}) - y) x_i$

Logistic Regression: Gradient Descent

- The input is two dimensional
- $P(y = 1|\boldsymbol{x}) = \frac{1}{1+e^{-(w_0 x_0 + w_1 x_1)}}$
- No bias b
- We can plot the decision boundary between the positive and negative class as when $w_0 x_0 + w_1 x_1$ is 0
  $P(y = 1|\boldsymbol{x}) = 0.5$
  - $w_0 x_0 + w_1 x_1 = 0$
  - $x_1 = -w_0 x_0 / w_1$



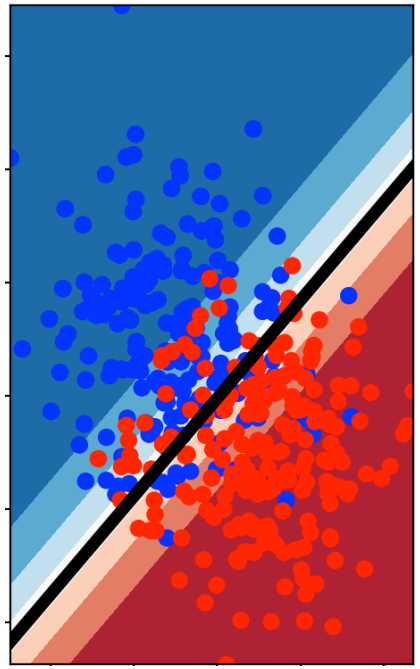
Classes Cannot Always be Perfectly Separated

- In many real-world applications, the classes are not perfectly separated
  - Data could be inherently noisy
  - The predictors may not be informative enough
  - The machine learning model may not be expressive enough
  - The training algorithm used may not be appropriate
- What could happen if your data contains more of one class than another?
  - For example, you want to learn if someone has a rare disease from medical tests
  - Since most people do not have the disease, most examples are of people that do not have the disease

Balanced vs Unbalanced Data

- One should always ensure that they balance their datasets!
  - Every gradient step can sample an equal number of states from each class
  - Or weight the contributions to the loss for each class to account for data being unbalanced
- Is this enough?

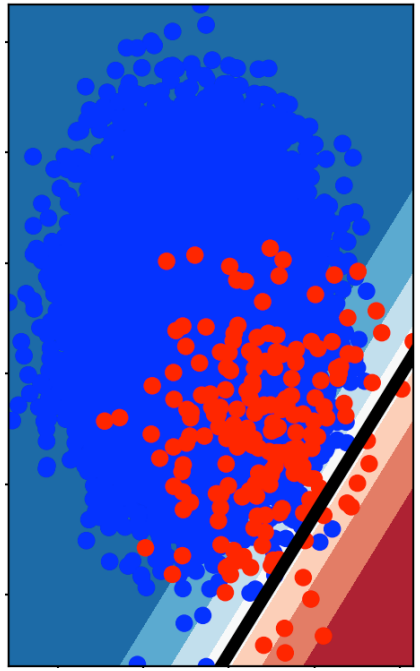
Decision boundary with balanced data



$$P(y = 1|\boldsymbol{x}) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + b)}}$$

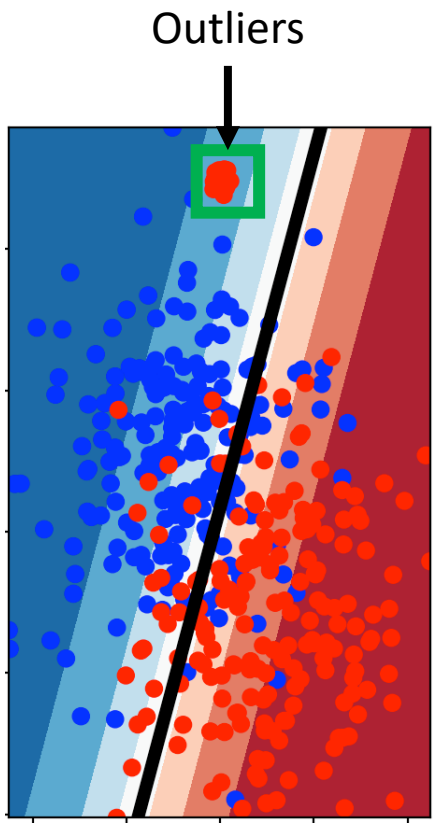
Has bias b

Decision boundary with unbalanced data



Balanced vs Unbalanced Data

- Even if the classes themselves are balanced, there may be outliers within those classes
- If these are not explicitly accounted for, the model may ignore them entirely
- For example, a rare disease that affects older people much more than children

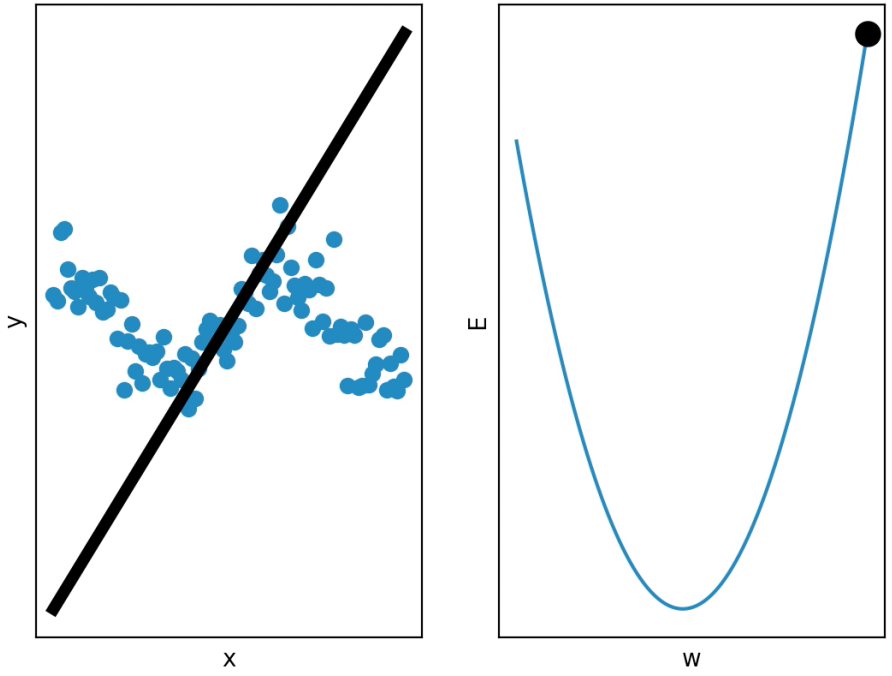


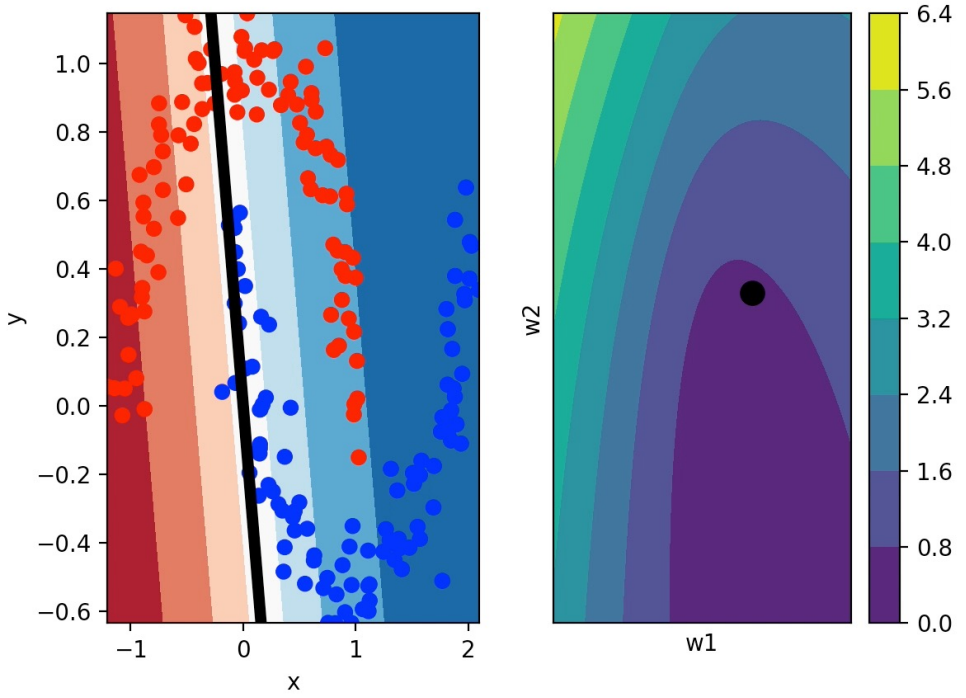
Softmax Regression

- If we have more than two classes, we can generalize logistic regression
- We got the logistic function from this equation $\frac{e^{\boldsymbol{w}_1^T \boldsymbol{x}}}{e^{\boldsymbol{w}_1^T \boldsymbol{x}} + e^{\boldsymbol{w}_0^T \boldsymbol{x}}}$
- If we have C classes, the probability of class i is $\frac{e^{\boldsymbol{w}_i^T \boldsymbol{x}}}{\sum_{j=1}^{C} e^{\boldsymbol{w}_j^T \boldsymbol{x}}}$
  - Known as a softmax function

Linear Models: Limitations

- Many interesting problems have a non-linear relationship between the inputs and outputs
- Linear models cannot handle these cases





Outline

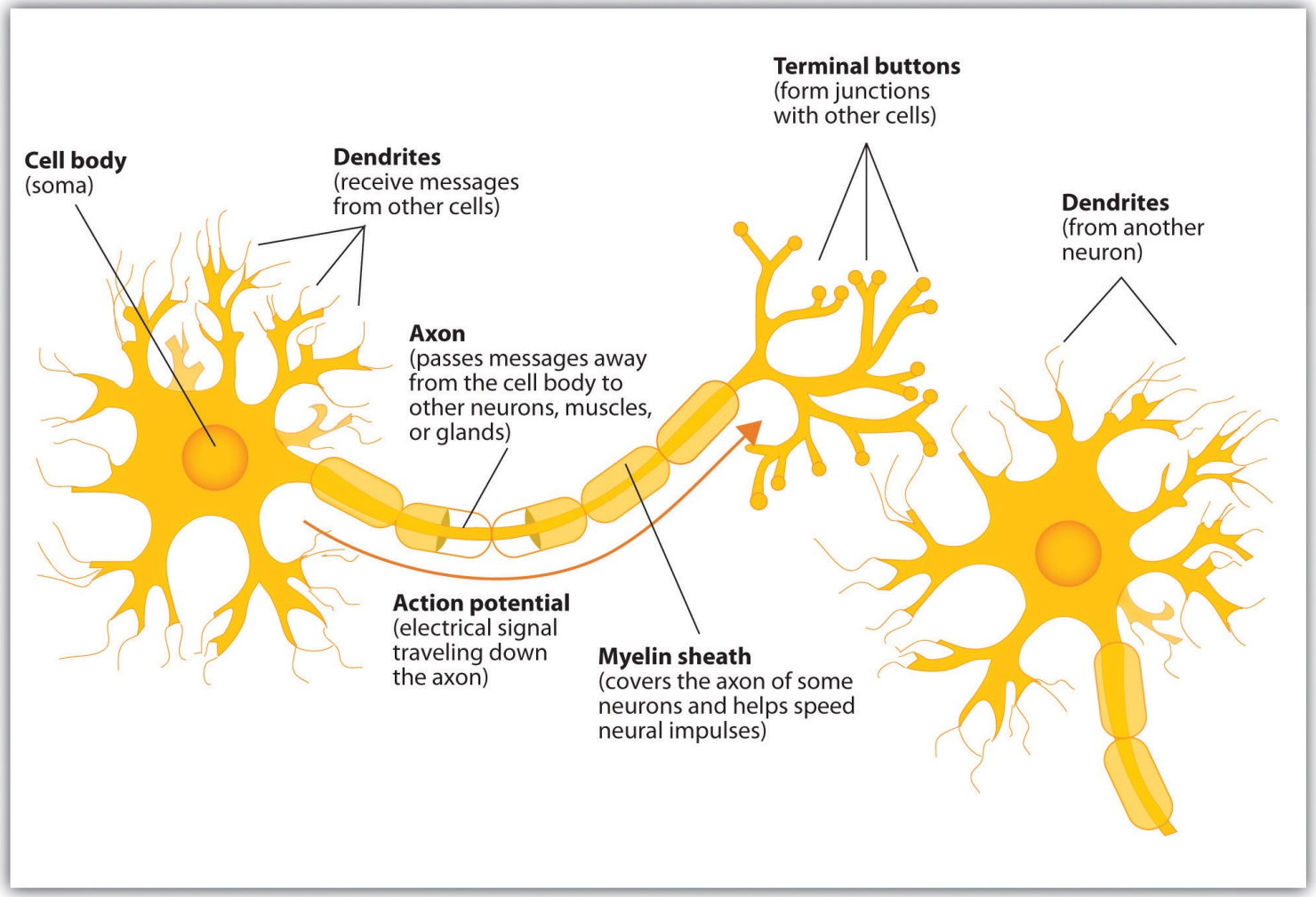
- Linear models
  - Linear regression
  - Logistic regression
- Neural networks
  - Architecture and backpropagation
  - Deep neural networks
  - Overfitting
  - Optimization

Machine Learning

- There are many different machine learning methods
  - Linear models
  - Deep neural networks
  - Support vector machines
  - Decision trees
  - K-nearest neighbors
- The type of model to use depends on your data
- Deep learning is often the best out of these methods when the data
  - High-dimensional
  - Low-level
  - Plentiful
  - Has a non-linear relationship between the input and the output

Artificial Neural Networks

- Inspired from biological neural networks
- Far from an exact model
- The main parallels are
  - Dendrites (inputs)
  - Action potential (activation function)
  - Axon terminals (outputs)

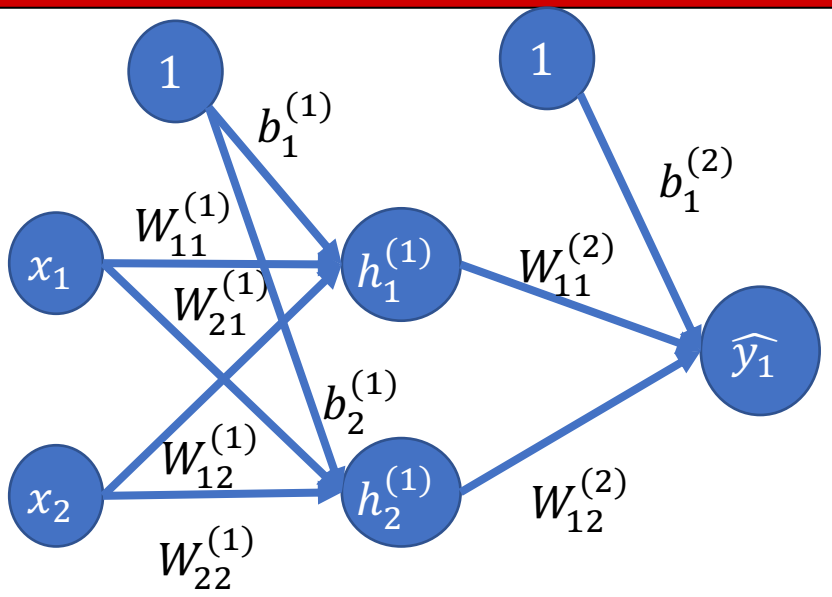


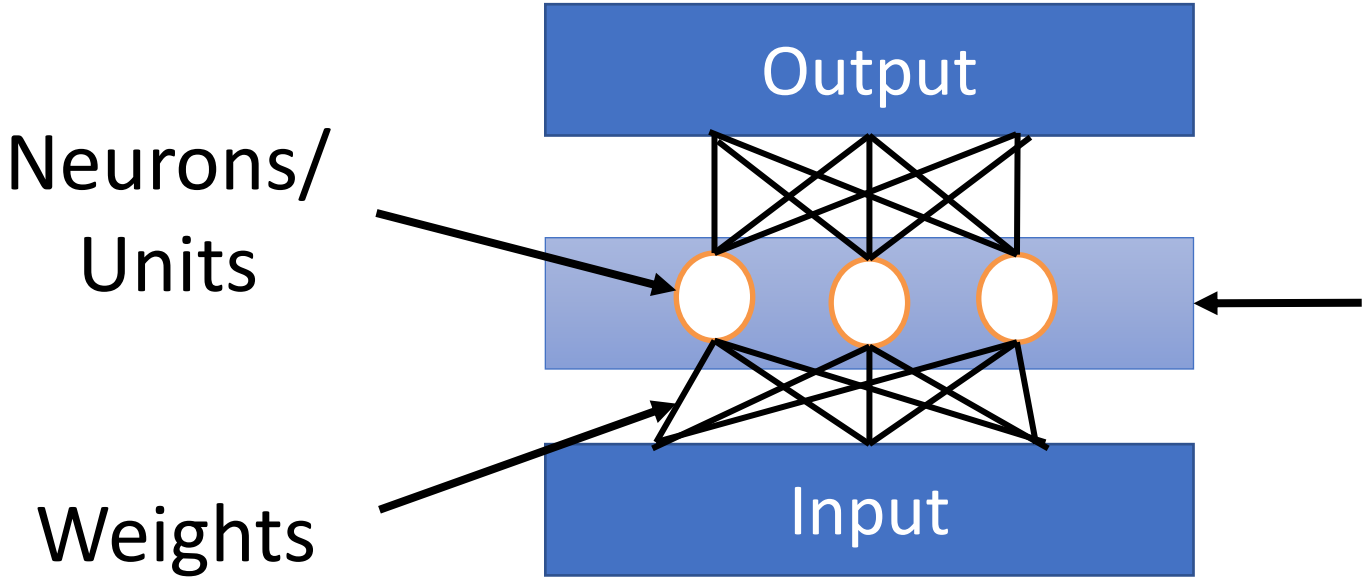
A biological neuron

Jennifer Walinga - https://opentextbc.ca/introductiontopsychology/chapter/3-1-the-neuron-is-the-building-block-of-the-nervous-system/

Neural Networks

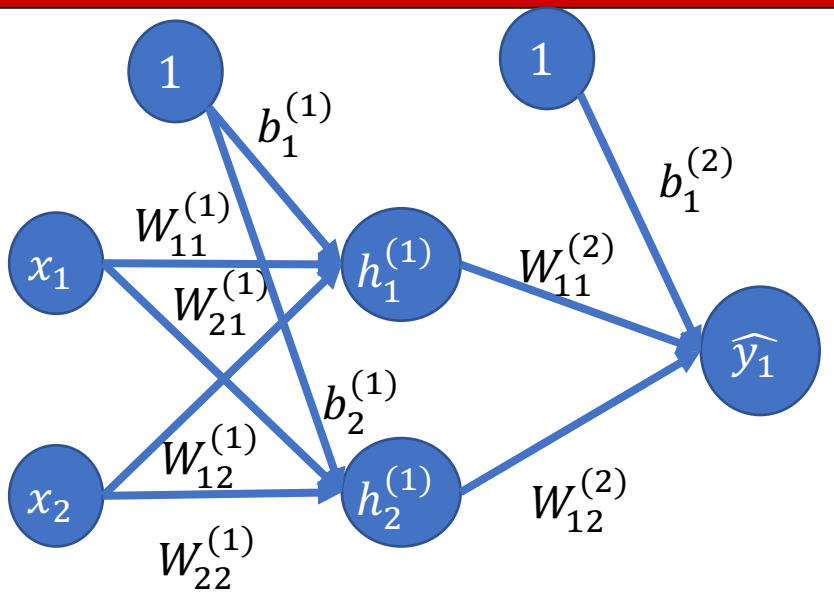
- $f(\boldsymbol{x}, \boldsymbol{w}) = \boldsymbol{W}^{(2)}\sigma(\boldsymbol{W}^{(1)}\boldsymbol{x})$
  - $\boldsymbol{h}^{(1)} = \sigma(\boldsymbol{W}^{(1)}\boldsymbol{x})$
  - $f(\boldsymbol{x}, \boldsymbol{w}) = \boldsymbol{W}^{(2)}\boldsymbol{h}^{1}$
- Where σ is some non-linear function





Quick Quiz: Linear Activation Functions

- $f(\boldsymbol{x}, \boldsymbol{w}) = \boldsymbol{W}^{(2)}\sigma(\boldsymbol{W}^{(1)}\boldsymbol{x})$
  - $\boldsymbol{h}^{(1)} = \sigma(\boldsymbol{W}^{(1)}\boldsymbol{x})$
  - $f(\boldsymbol{x}, \boldsymbol{w}) = \boldsymbol{W}^{(2)}\boldsymbol{h}^{1}$
- What if σ is a linear function?



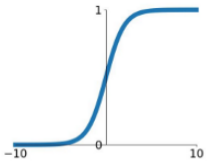
- The resulting function would also be linear. This is true no matter how deep the network is.

Backpropagation: Activation Functions

- Allow neural network to learn non-linear functions
- Logistic (Sigmoid)
  - $\sigma(x) = \frac{1}{1+e^{-x}}$
  - $\sigma'(x) = \sigma(x)(1-\sigma(x))$
- Rectified Linear Unit (ReLU)
  - $\sigma(x) = \max(0, x)$
  - $\sigma'(x) = 0$ if $x \leq 0$
  - $\sigma'(x) = 1$ if $x > 0$
  - Derivative undefined at zero but does not matter in practice
- Activation functions can also be parameterized and learned through gradient descent

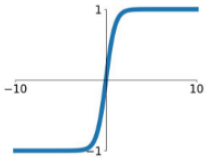
Sigmoid

$\sigma(x) = \frac{1}{1+e^{-x}}$



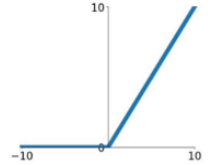
tanh

$\tanh(x)$



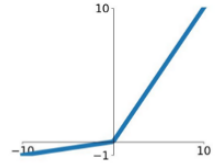
ReLU

$\max(0, x)$



Leaky ReLU

$\max(0.1x, x)$

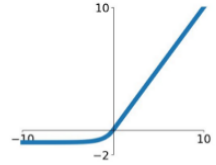


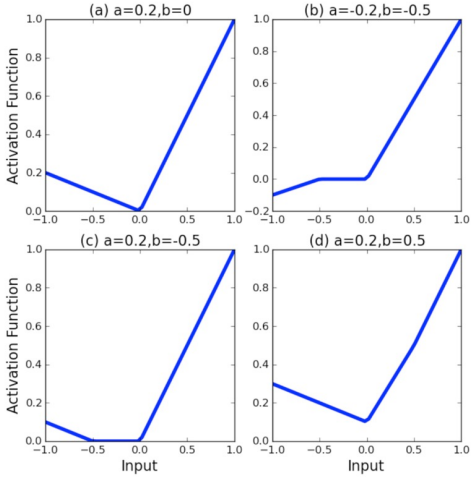
Maxout

$\max(w_1^T x + b_1, w_2^T x + b_2)$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$





Adaptive piecewise linear units

https://medium.com/@shrutijadon10104776/survey-on-activation-functions-for-deep-learning-9689331ba092

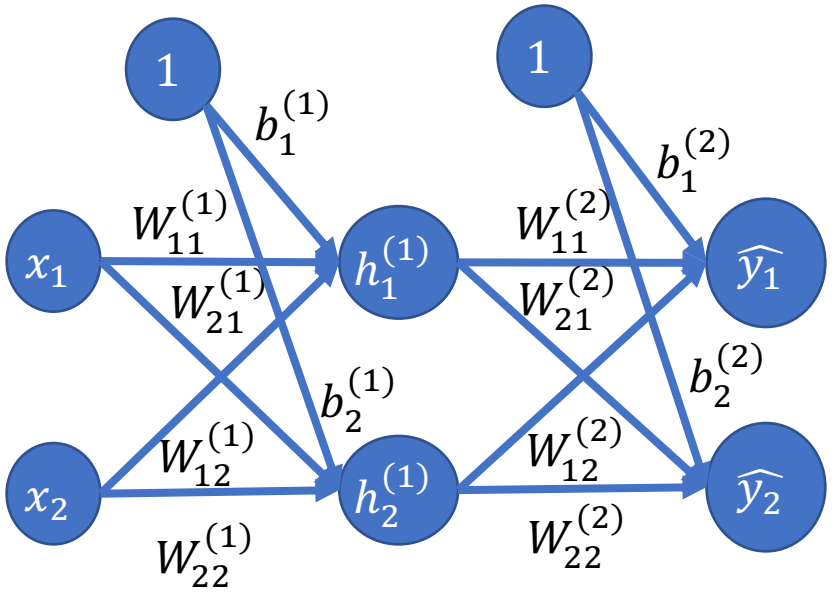
Agostinelli, F., Hoffman, M., Sadowski, P., & Baldi, P. (2014). Learning activation functions to improve deep neural networks. *arXiv preprint arXiv:1412.6830*.

Neural Networks: Universal Function Approximation

- Given enough hidden units, neural networks can approximate any function with arbitrary precision
- Cannot guarantee convergence

Backpropagation

- We do gradient descent via backpropagation
  - Just the application of the chain rule
- $\boldsymbol{h}^{(1)} = \sigma(\boldsymbol{W}^{(1)}\boldsymbol{x})$
  - $h_j^{(1)} = \sigma(\sum W_{jk}^{(1)} x_k)$
- $f(\boldsymbol{x}, \boldsymbol{w}) = \boldsymbol{W}^{(2)}\boldsymbol{h}^{(1)} = \hat{\boldsymbol{y}}$
  - $\hat{y}_i = \sum_j W_{ij}^{(2)} h_j^{(1)}$
- $E(\boldsymbol{w}) = \frac{1}{2}\|(\boldsymbol{y} - \hat{\boldsymbol{y}})\|_2^2 = \frac{1}{2}\|\boldsymbol{e}\|_2^2$
  - $= \frac{1}{2}\sum_i (y_i - \hat{y}_i)^2 = \frac{1}{2}\sum_i e_i^2$
- $\frac{\partial E(\mathbf{w})}{\partial W_{ij}^{(2)}} = \frac{\partial E(\mathbf{w})}{\partial \boldsymbol{e}} \frac{\partial \boldsymbol{e}}{\partial \hat{\boldsymbol{y}}} \frac{\partial \hat{\boldsymbol{y}}}{\partial W_{ij}^{(2)}} = -(y_i - \hat{y}_i) h_j$
- $\frac{\partial E(\mathbf{w})}{\partial W_{jk}^{(1)}} = \frac{\partial E(\mathbf{w})}{\partial \boldsymbol{e}} \frac{\partial \boldsymbol{e}}{\partial \hat{\boldsymbol{y}}} \frac{\partial \hat{\boldsymbol{y}}}{\partial \boldsymbol{h}^{(1)}} \frac{\partial \boldsymbol{h}^{(1)}}{\partial W_{jk}^{(1)}} = \sum_i -(y_i - \hat{y}_i) W_{ij}^{(2)} \sigma'(\sum W_{jk}^{(1)} x_k)\, x_k$

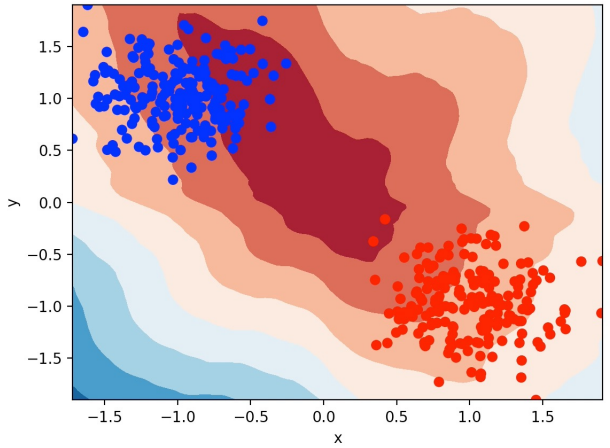


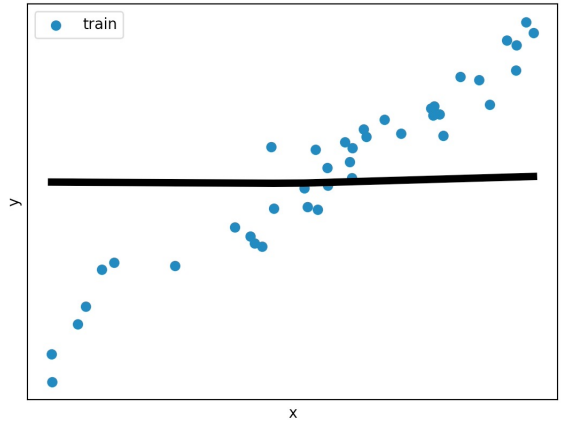
Neural Networks: Regression and Classification

Classification

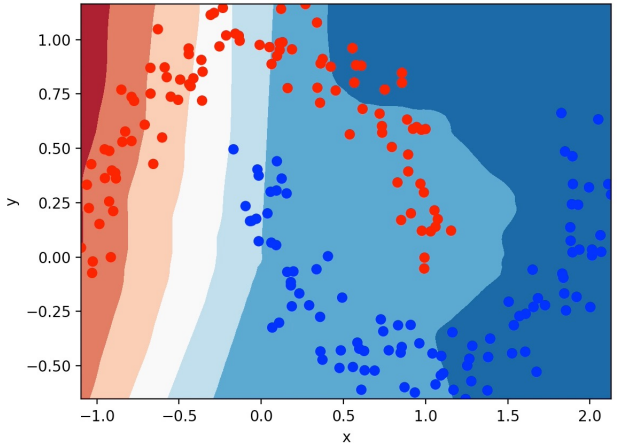
Regression

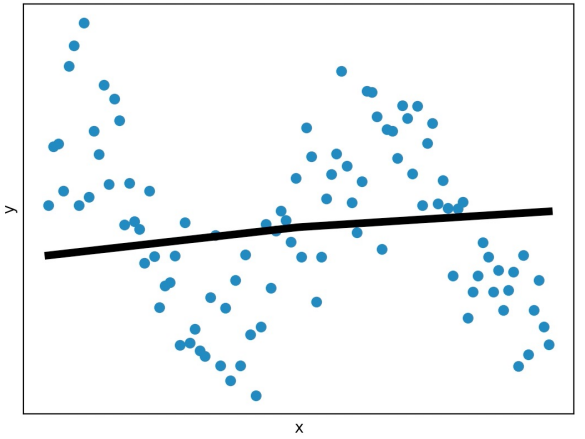
Linear





Non-Linear

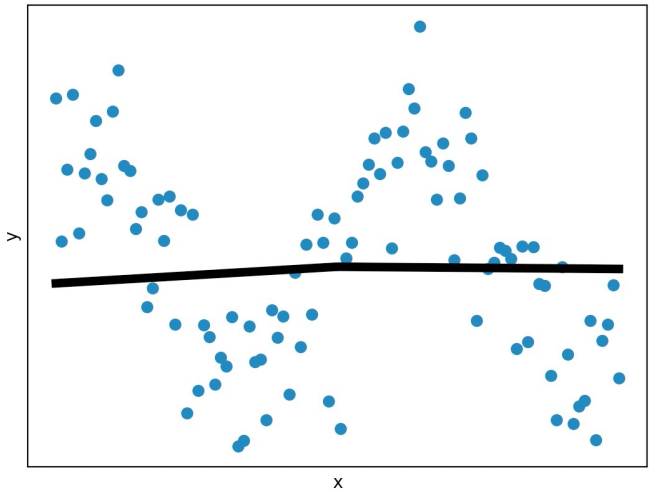




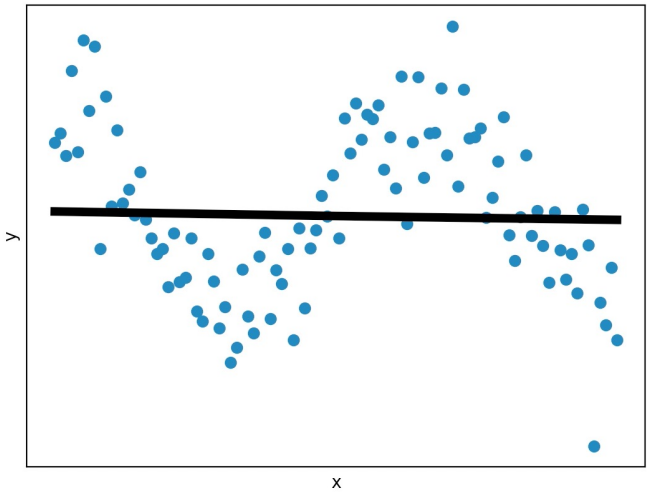
Hyperparameters

- Parameters are learned from the data
- Hyperparameters are set before training
- Learning rate
  - Defines how large the steps will be during gradient descent
  - Usually denoted by α
- Number of neurons
  - How “wide” the neural network is
- Many more!

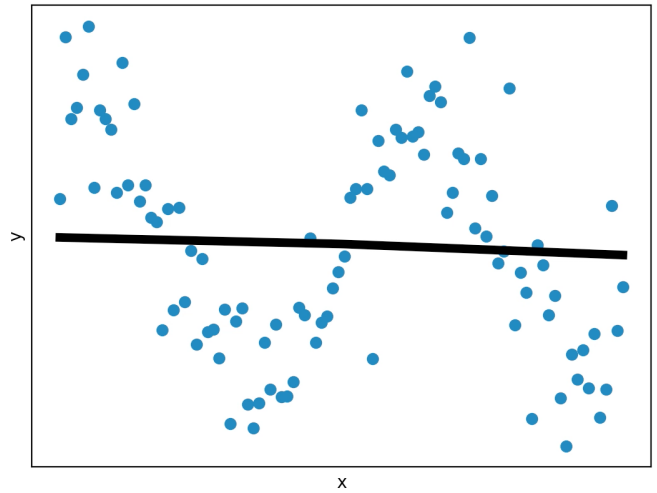
Quick Quiz: What are the Best Hyperparameters?



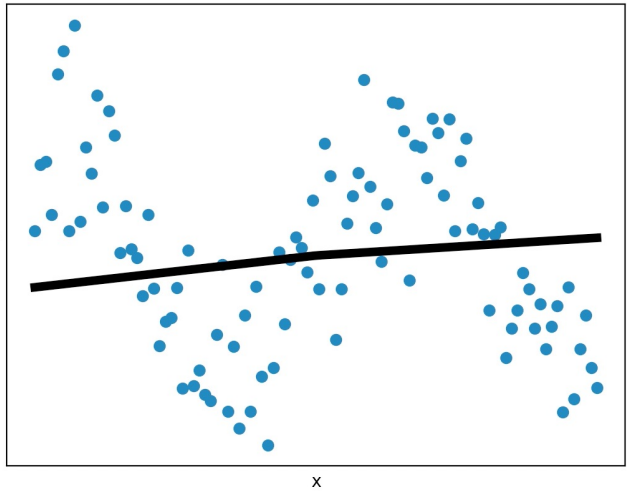
$\alpha = 0.1$, neurons = 100



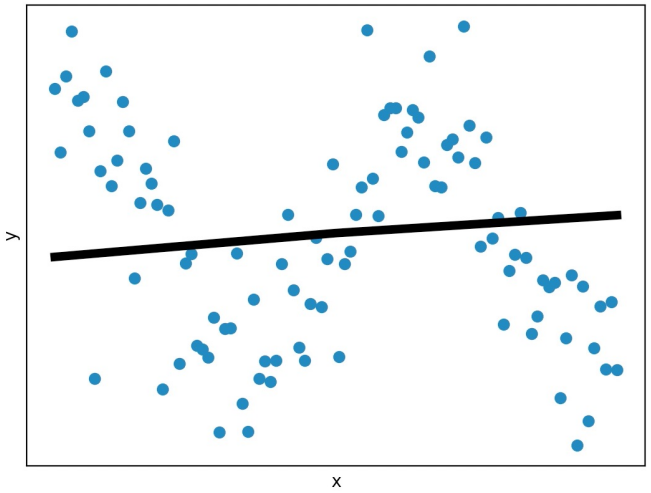
$\alpha = 0.25$, neurons = 100



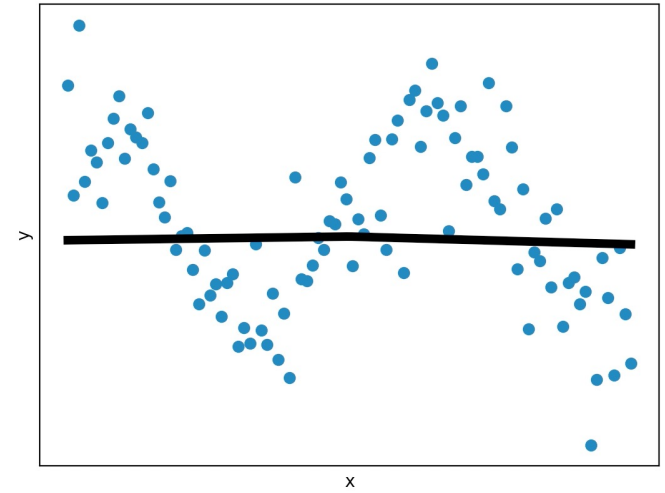
$\alpha = 0.75$, neurons = 100



$\alpha = 0.1$, neurons = 1000



$\alpha = 0.25$, neurons = 1000



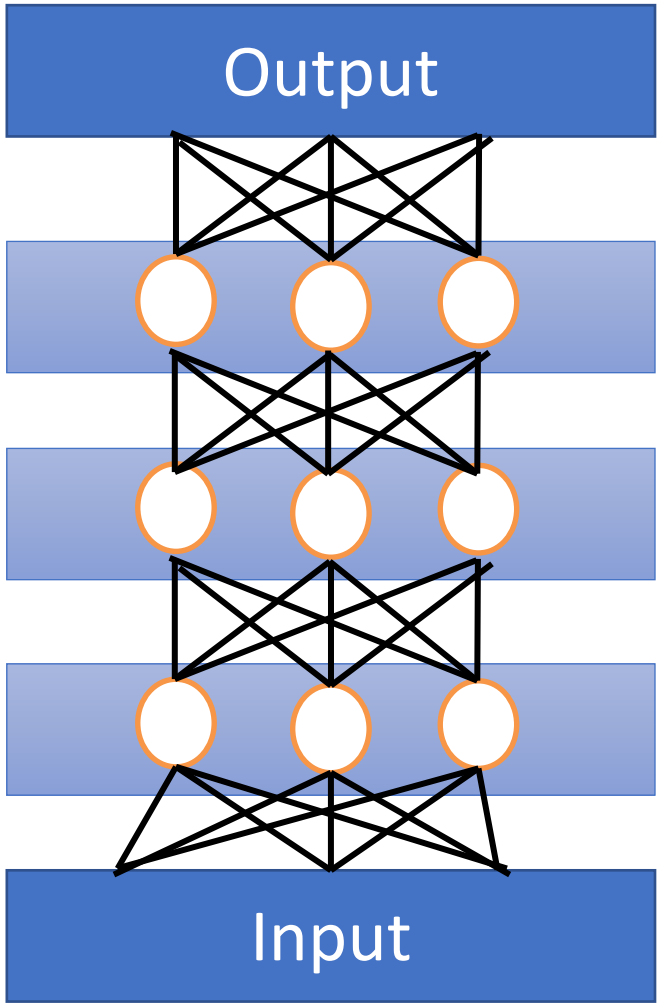
$\alpha = 0.75$, neurons = 1000

Outline

- Linear models
  - Linear regression
  - Logistic regression
- Neural networks
  - Architecture and backpropagation
  - Deep neural networks
  - Overfitting
  - Optimization

Deep Neural Networks

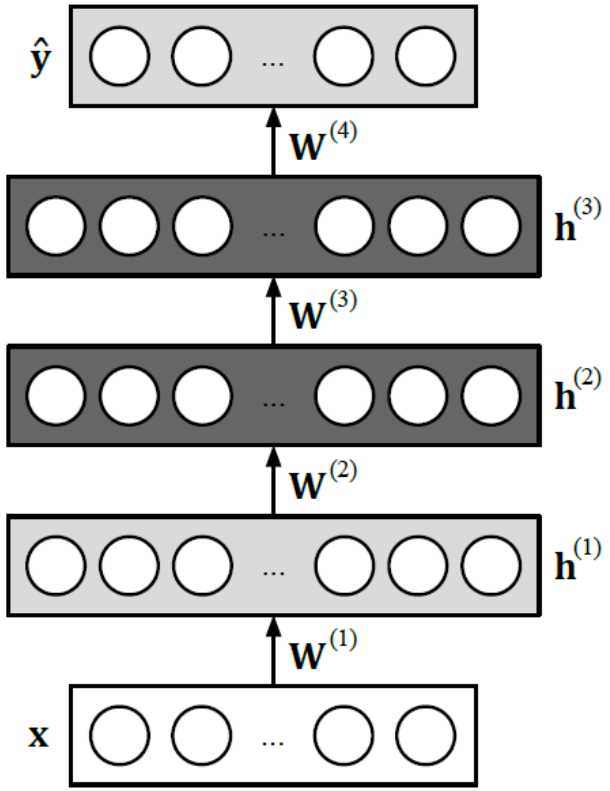
- Stack hidden layers to obtain a deep neural network
- “Deep learning allows computational models that are composed of multiple processing layers to learn representations of data with multiple levels of abstraction.”

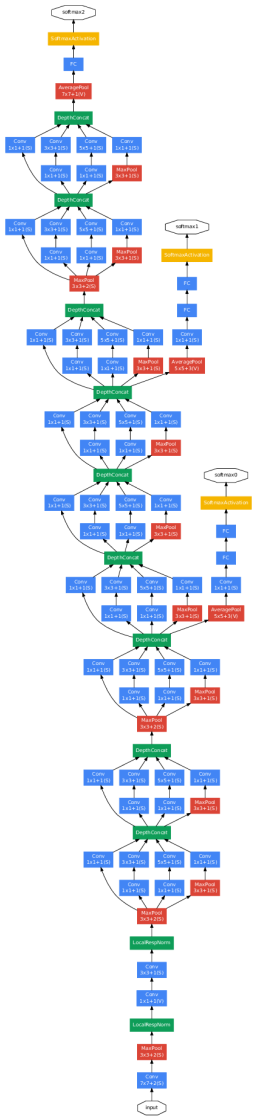


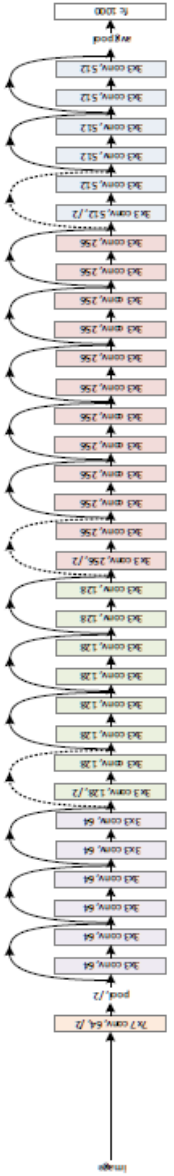
LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. nature, 521(7553), 436.

Deep Neural Networks

- There are a wide variety of ways one can create a deep neural network

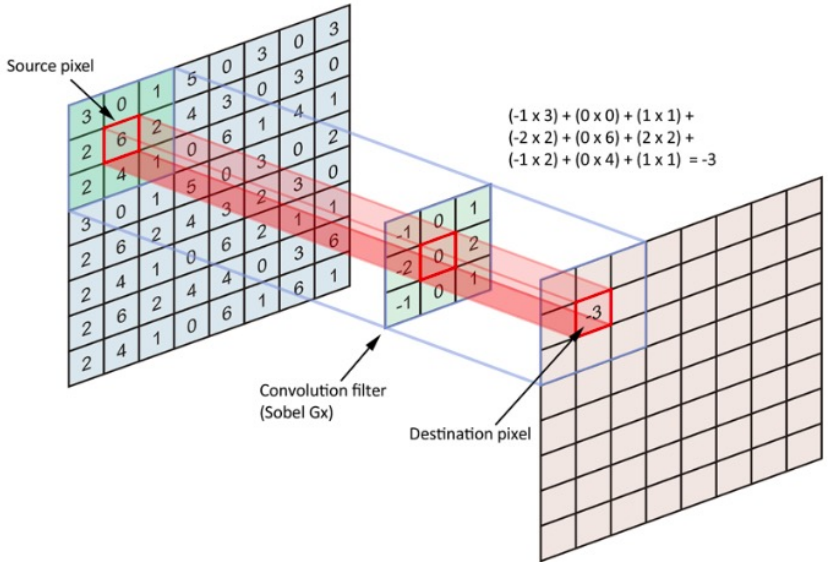


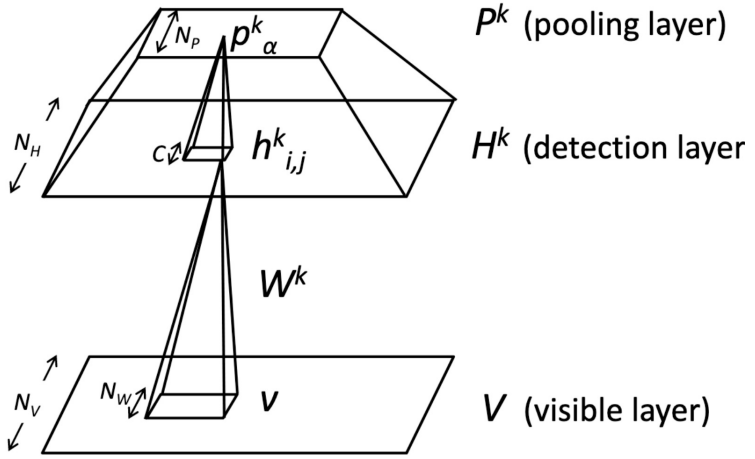




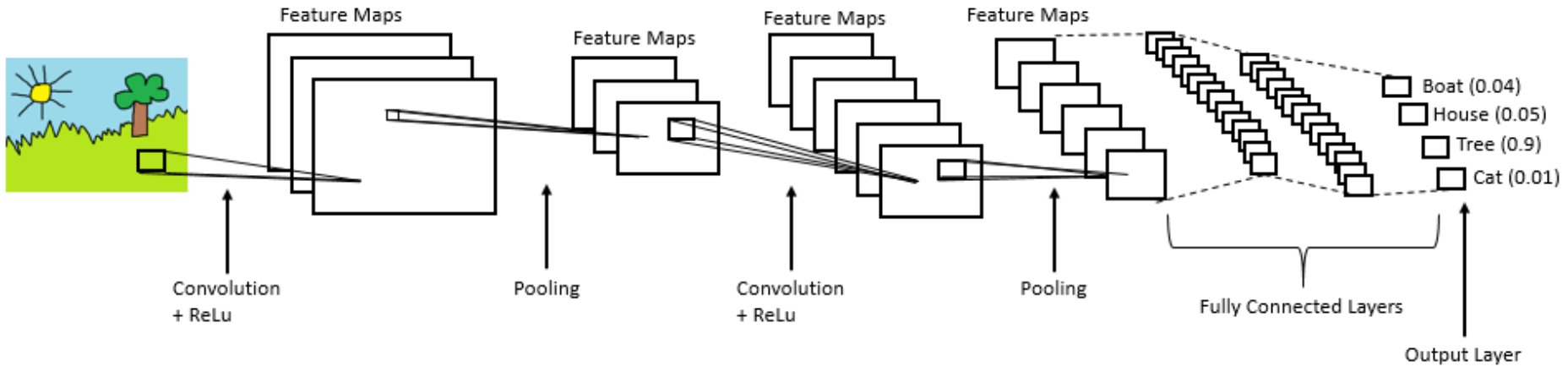
Convolution and Pooling

- We can design neural networks to take advantage of structured data, such as images
- Convolution helps to add translation invariance
  - Strong inductive bias
- Pooling shrinks the representation, allowing subsequent layers to focus on higher-level information and have a larger receptive field



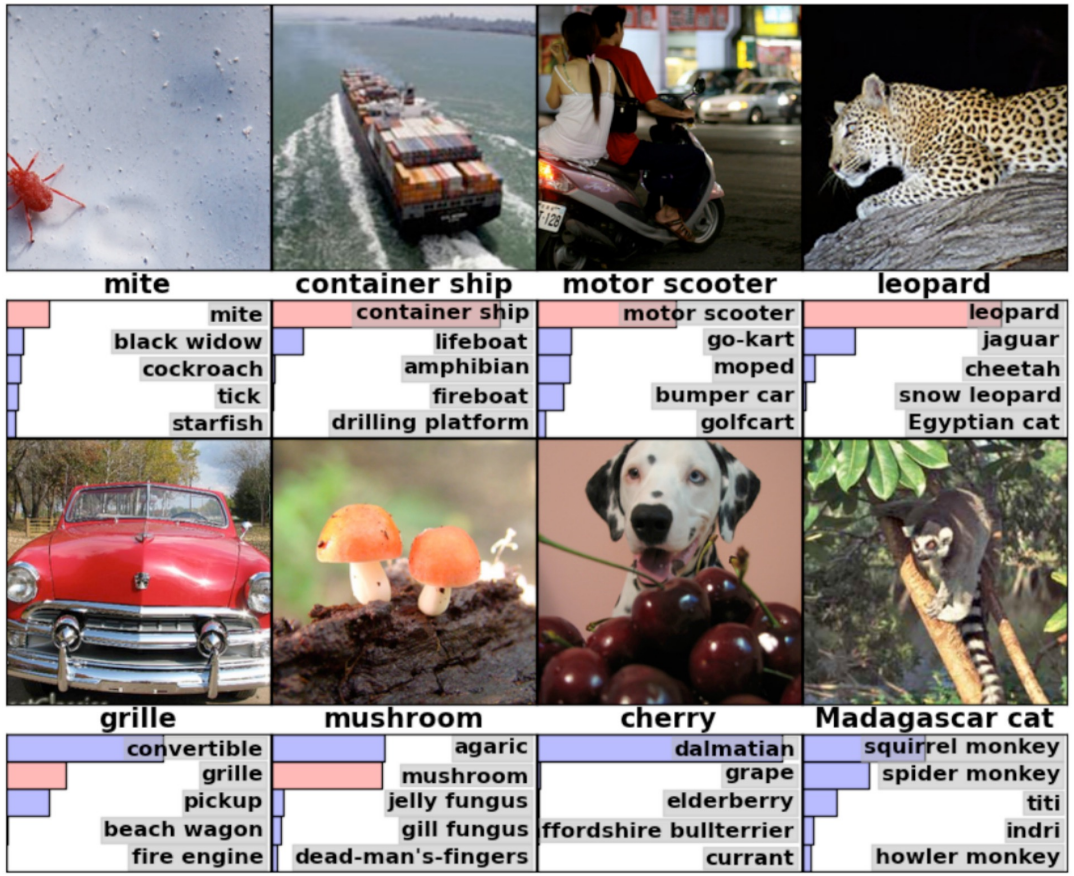


Convolution and Pooling



Convolution and Pooling

- Was central to the breakthrough on the Imagenet dataset



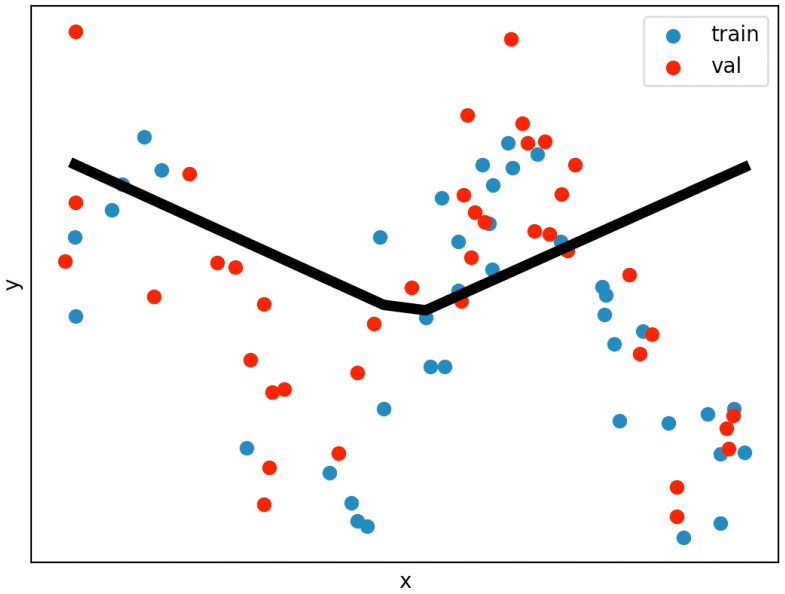
Outline

- Linear models
  - Linear regression
  - Logistic regression
- Neural networks
  - Architecture and backpropagation
  - Deep neural networks
  - Overfitting
  - Optimization

Overfitting/Regularization

- **Training Dataset**: Used to train neural network
- **Validation Dataset**: Not used to train neural network. Used to determine how well neural network generalizes
- **Test Dataset:** Only for seeing the final performance of neural network. Not used for training or validation.

Loss – Train: 0.042, Val: 0.35

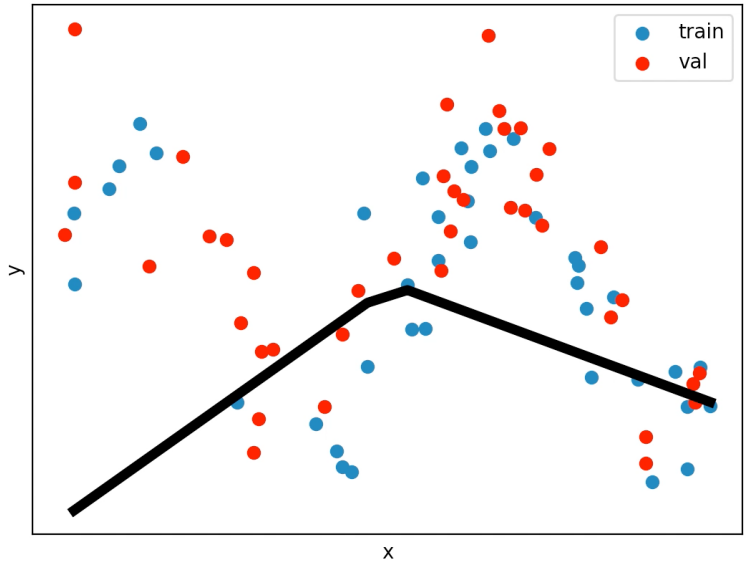


DNN 5 hidden layers and 500 neurons per layer

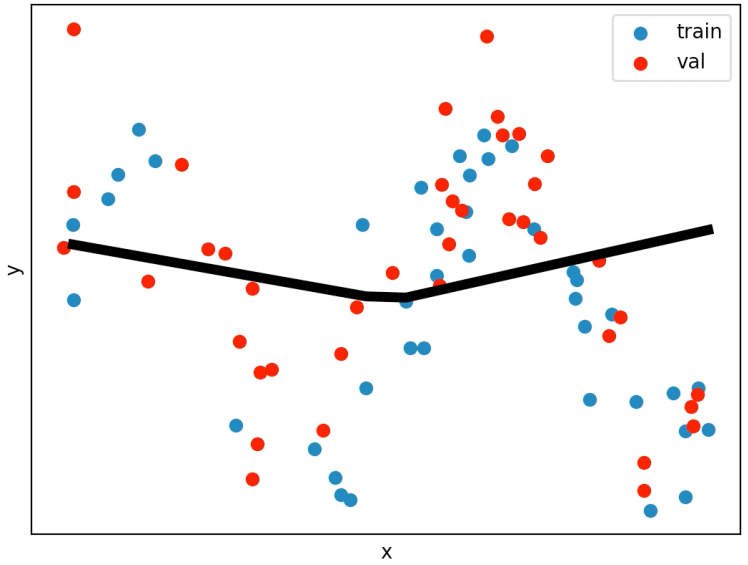
Regularization: Weight Regularization

- Weight regularization
  - $E(\boldsymbol{w}) = \frac{1}{2n}\sum_n ||\boldsymbol{y}_n - \hat{\boldsymbol{y}}_n||_2^2 + \lambda \sum_l \sum_i \sum_j W_{ij}^2$
  - Make the weights less "sensitive" to the input

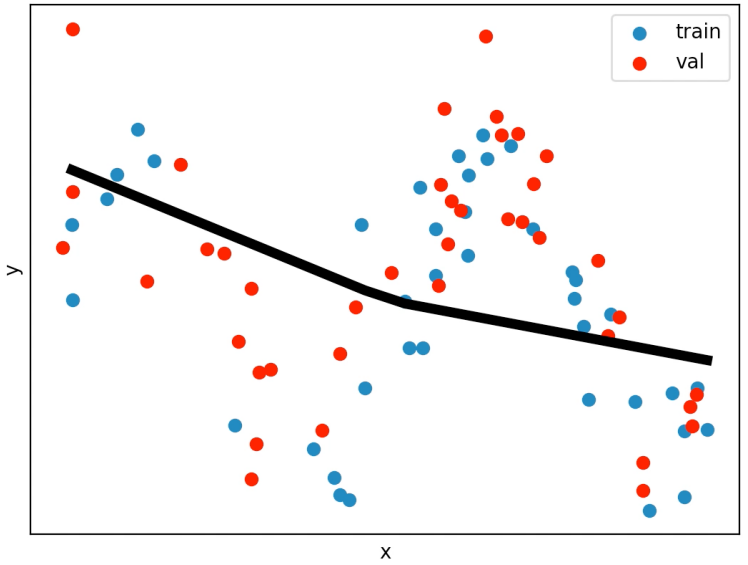
Loss – Train: 0.12, Val: 0.29, $\lambda = 0.01$



Loss – Train: 0.18, Val: 0.26, $\lambda = 0.05$

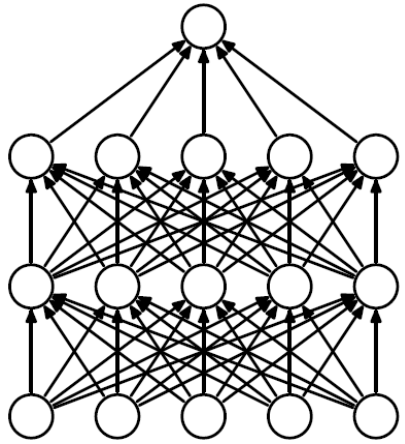


Loss – Train: 0.70, Val: 0.77, $\lambda = 0.2$

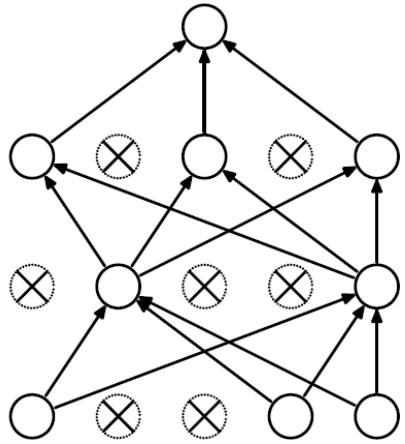


Overfitting/Regularization: Dropout

- Dropout
  - Randomly drop connections between neurons during training

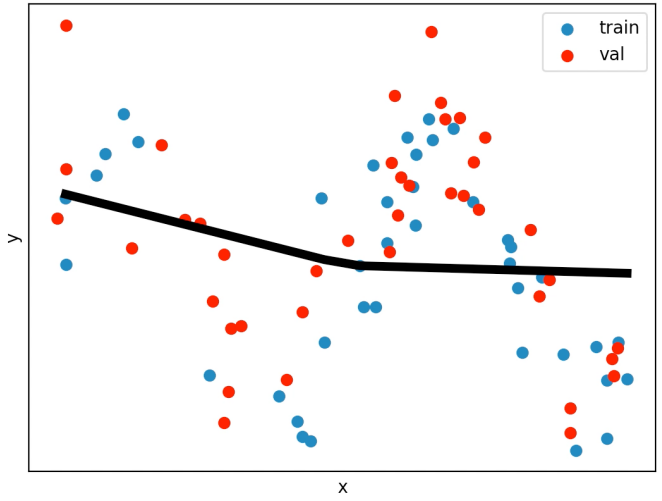


(a) Standard Neural Net

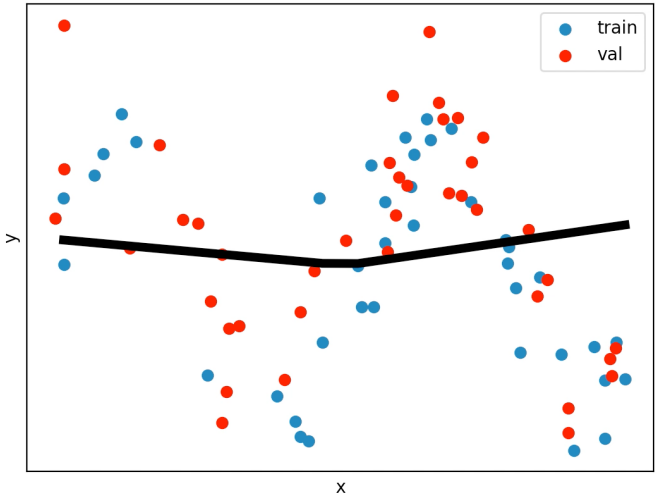


(b) After applying dropout.

Loss – Train: 0.23, Val: 0.25, $p = 0.5$



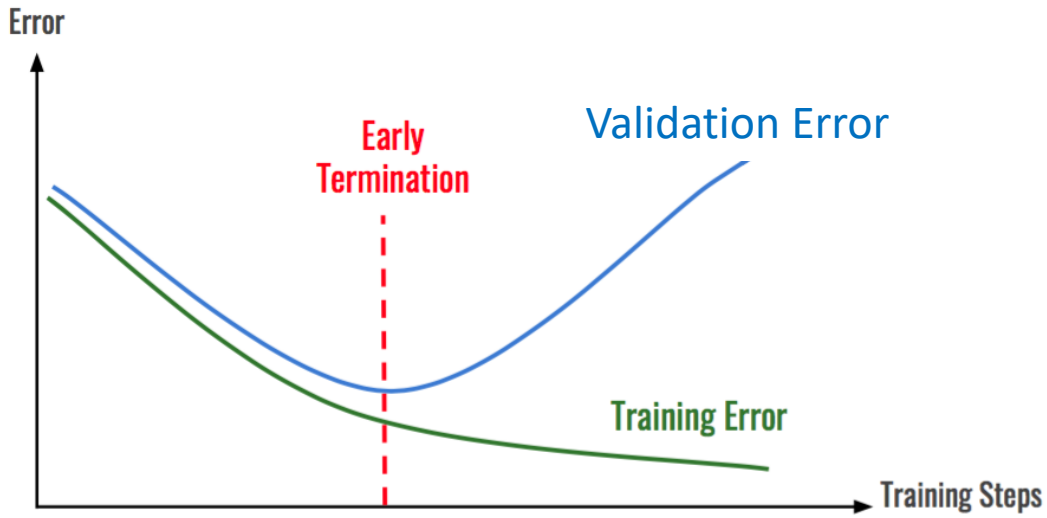
Loss – Train: 0.70, Val: 0.75, $p = 0.9$



Regularization: Add More Data

- Harder to overfit if there is more data
- Collect more data
- Augment current data
  - Rotations, flipping, translations
  - Adding noise

Overfitting/Regularization

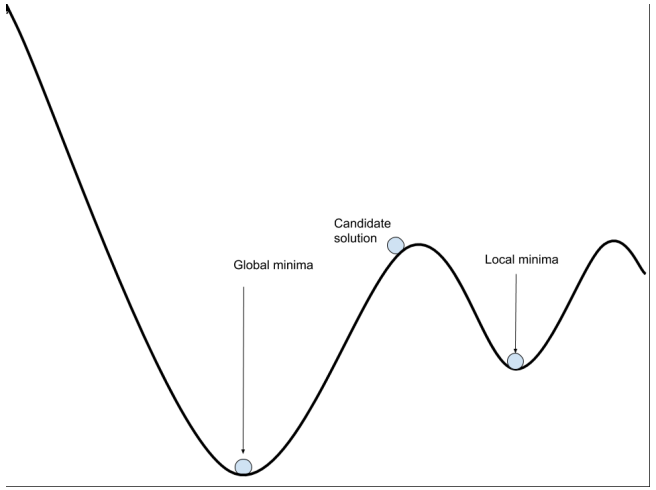


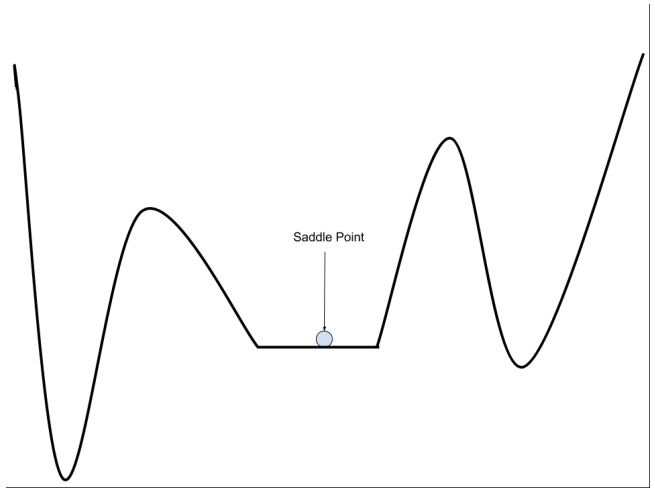
Outline

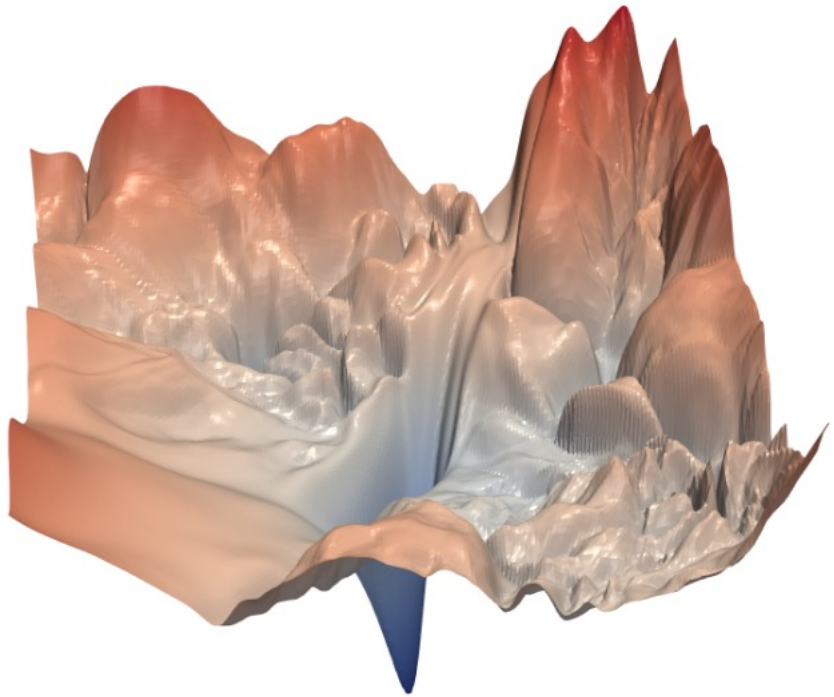
- Linear models
  - Linear regression
  - Logistic regression
- Neural networks
  - Architecture and backpropagation
  - Deep neural networks
  - Overfitting
  - Optimization

Optimization: Loss Surface

- No longer convex
- Local minima
- Saddle points



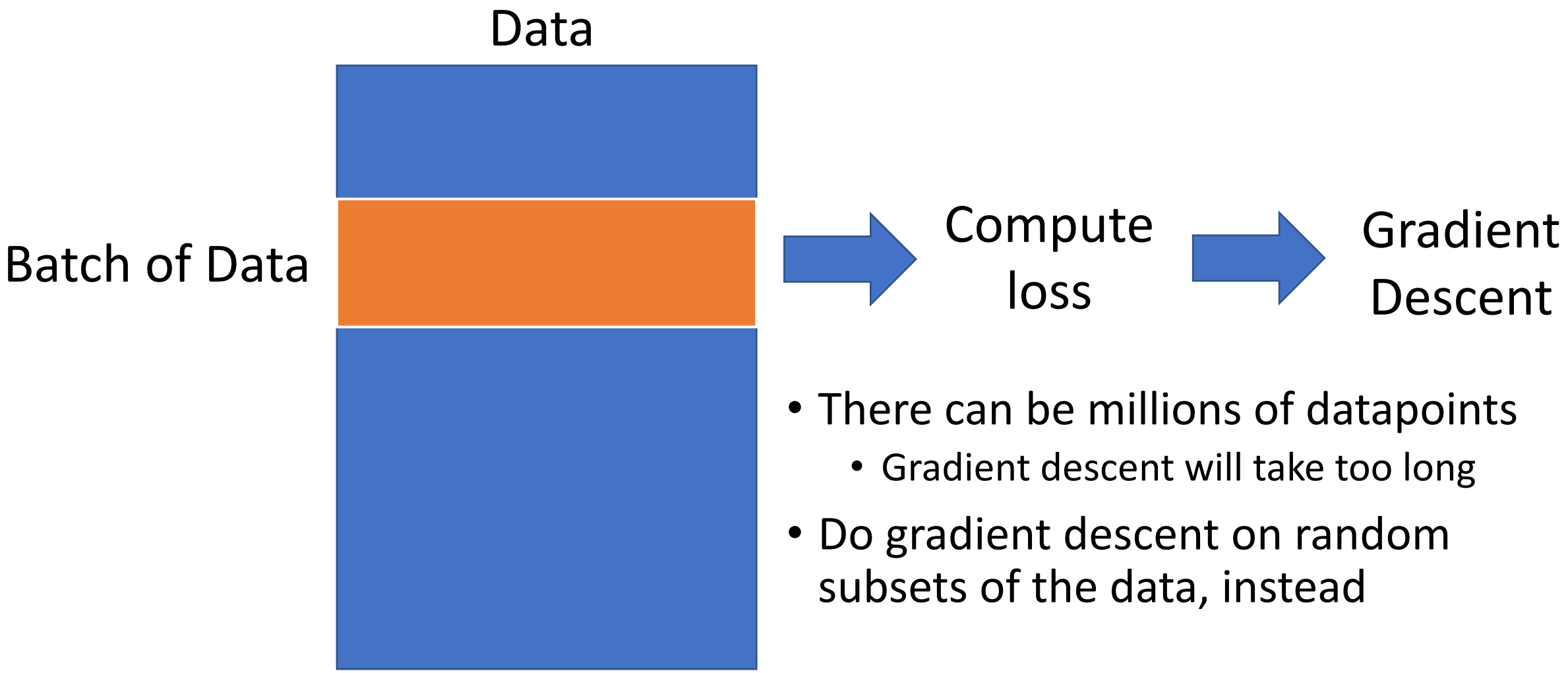




Optimization: Gradient Based Methods

- Vanilla Gradient Descent
  - $\boldsymbol{w} = \boldsymbol{w} - \alpha \nabla_{\mathbf{w}} \mathrm{E}(\mathbf{w})$
- Gradient Descent with Momentum
  - $\boldsymbol{v} = \mu \boldsymbol{v} + \alpha \nabla_{\mathbf{w}} \mathrm{E}(\mathbf{w})$
  - $\boldsymbol{w} = \boldsymbol{w} - \boldsymbol{v}$
- ADAM
  - $\boldsymbol{m} = \beta_1 \boldsymbol{m} + (1 - \beta_1)(\nabla_{\mathbf{w}} \mathrm{E}(\mathbf{w}))^2$ //estimate of the mean of the gradients
  - $\boldsymbol{v} = \beta_2 \boldsymbol{v} + (1 - \beta_2)(\nabla_{\mathbf{w}} \mathrm{E}(\mathbf{w}))^2$ //estimate of the variance of the gradients
  - $\hat{\boldsymbol{m}}$ and $\hat{\boldsymbol{v}}$ are bias corrected estimates of the mean and variance
  - $\boldsymbol{w} = \boldsymbol{w} - \frac{\alpha}{\sqrt{\hat{\boldsymbol{v}}} + \epsilon} \hat{\boldsymbol{m}}$
- Many others: https://ruder.io/optimizing-gradient-descent/

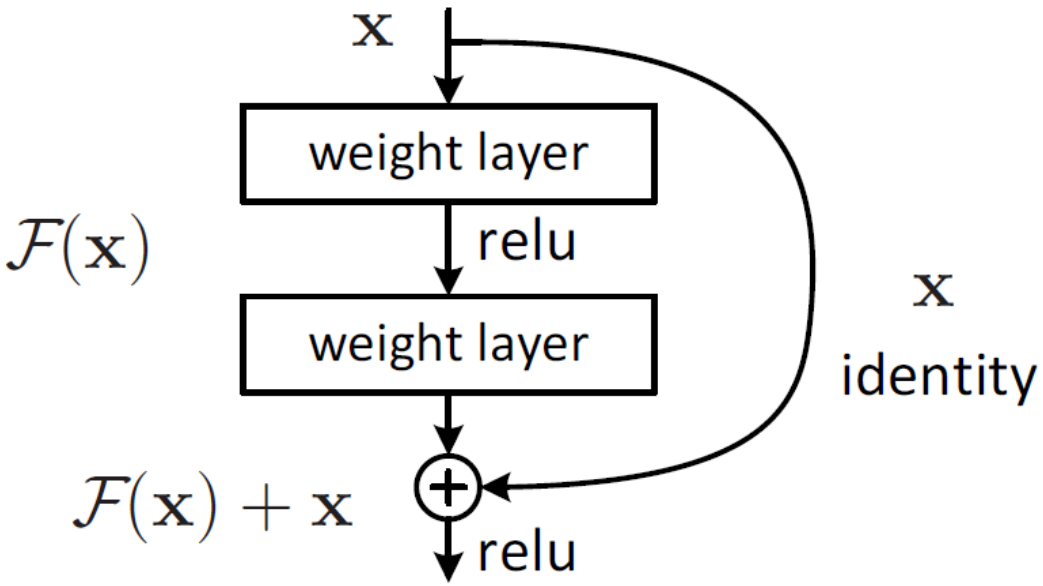
Optimization: Stochastic Gradient Descent

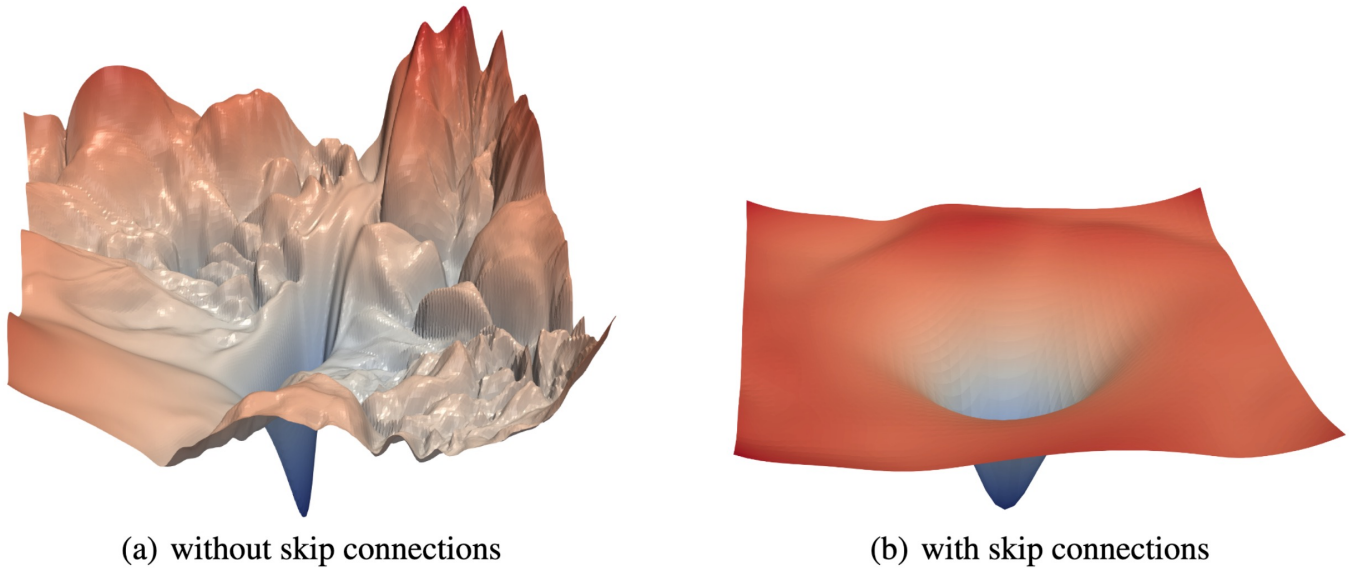


- There can be millions of datapoints
  - Gradient descent will take too long
- Do gradient descent on random subsets of the data, instead

Optimization: Residual Neural Networks

- Training can become more difficult as the number of layers increases
- Adding skip connections allows us to train networks with hundreds of layers



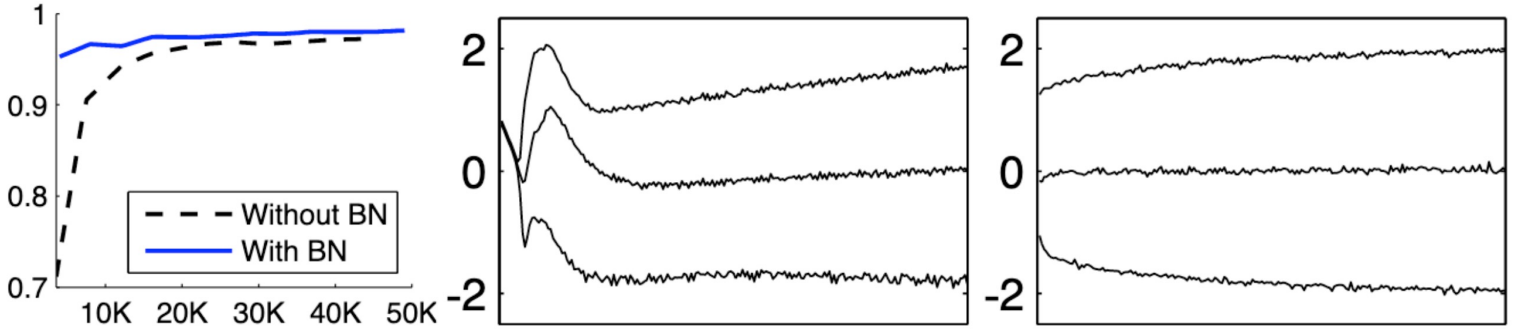


(a) without skip connections (b) with skip connections

Loss surface

Optimization: Batch Normalization

- Normalizes the input to the activation function to have a of mean 0 and standard deviation of 1
- Stabilizes training
  - Allows larger learning rates
  - Reduces importance of initialization
- $\boldsymbol{H} = \sigma(BN(\boldsymbol{W}\boldsymbol{X}))$
- Adds some regularization



(a) (b) Without BN (c) With BN

Optimization: Initialization

- The weights of the DNN are randomly initialized
- Initialization can play a large role in optimization
- Xavier/Glorot initialization is fairly common
- Initialization matters less when doing
  - Batch normalization
  - Weight normalization

What to Try?

- Activation Function
  - Rectified Linear Units
- Gradient-Based Optimization
  - SGD with momentum
  - ADAM
- Convolution (for structured input like images or sound)
- Batch Normalization
- Residual Networks
- If overfitting?
  - Weight regularization
  - Dropout
    - In higher layers first

Deep Learning/Machine Learning Demos

- https://p.migdal.pl/interactive-machine-learning-list/
- https://cs.stanford.edu/people/karpathy/convnetjs/demo/regression.html

Relevant Papers

- **Xavier/Glorot Init:** Glorot, Xavier, and Yoshua Bengio. "Understanding the difficulty of training deep feedforward neural networks." *Proceedings of the thirteenth international conference on artificial intelligence and statistics*. 2010.
- **SGD w/ Momentum:** Sutskever, Ilya, et al. "On the importance of initialization and momentum in deep learning." *International conference on machine learning*. 2013.
- **Imagenet:** Krizhevsky, Alex, Ilya Sutskever, and Geoffrey E. Hinton. "Imagenet classification with deep convolutional neural networks." *Advances in neural information processing systems*. 2012.
- **ADAM:** Kingma, Diederik P., and Jimmy Ba. "Adam: A method for stochastic optimization." *arXiv preprint arXiv:1412.6980* (2014).
- **Batch Normalization:** Ioffe, Sergey, and Christian Szegedy. "Batch normalization: Accelerating deep network training by reducing internal covariate shift." *arXiv preprint arXiv:1502.03167* (2015).
- **Residual Networks**: He, Kaiming, et al. "Deep residual learning for image recognition." *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016.